

Resume



안녕하세요 이영민입니다.

Email

bbok4yo@gmail.com

Channel

[Github](#)

[Velog](#)

ABOUT ME

- React와 Next.js App Router로 동적 라우팅과 전역 상태 관리를 포함한 화면 아키텍처를 설계합니다.
- TypeScript로 타입을 정의하고 컴포넌트 재사용 구조와 API 호출 시점을 서비스 관점에서 설계합니다.
- 백엔드 데이터 구조와 API 연동을 이해하고 프론트엔드 상태 관리 전략을 함께 설계합니다.
- Git과 PR 기반 협업에서 기획 의도를 확인하고 코드 리뷰에 참여합니다.
- AI와 협업해 요구사항 분석부터 SDD 설계, 자동화된 코드 품질 검증까지 개발 흐름을 연결합니다.

CAREERS

리얼라인

기간: 2026.01 ~ 2026.02

- AI 기반 소프트웨어 엔지니어링 파이프라인 구축: 요구사항 분석·SDD 설계·구현·품질 검증 전 과정에 Claude Code를 붙여 개발 주기를 줄였습니다.
- 단기 다중 프로젝트 완수: 1개월 내 전략 보고서 생성기·지도 데이터 시각화·스팟업 웹 3종을 스펙 기반 AI 워크플로우로 구현했습니다.

성과: AI를 활용한 일관된 개발 및 품질 엔지니어링

1. AI-Assisted SDD 개발 및 문서화

- 아키텍처 설계: 요구사항을 바탕으로 Claude Code와 상호작용하며 시스템 아키텍처, 데이터 모델링, 컴포넌트 계층을 정의하는 SDD를 신속하게 작성했습니다.
- 문서화 자동화: API 명세 변경과 상태 관리 플로우를 AI로 문서에 실시간 반영해 최신 명세를 바로 확인하도록 했습니다.

2. Custom Skill·Hook 기반 코드 품질 엔지니어링

- 자동화 품질 검증: Custom Skill과 Git Hook(Pre-commit/Pre-push)을 연동하여 커밋 단계에서 AI가 정적 분석·컨벤션 검사를 자동 수행하도록 구성했습니다.
- 지속적 리팩토링: AI가 프로젝트 전체 컨텍스트를 바탕으로 보안 취약점 점검, 중복 코드 제거, 아키텍처 개선안을 제안하고 적용했습니다.

라운피플

기간: 2024.01 ~ 2024.04

성과: 테스트 케이스(TC) 및 주어진 테스트 항목에 기반한 기능 테스트 수행

- JIRA를 활용해 버그 리포트를 생성하고 이슈를 트래킹했습니다.
- 테스트 결과 보고서를 작성하고 개발팀에 피드백을 전달했습니다.
- 관련 기술: JIRA

ACTIVITIES

프론트엔드 개발자 과정

소속/기관: Kakao x goorm

활동 연도: 2024.07~2025.04

- Next.js를 학습하고 실제 프로젝트에 적용해 실전 감각을 키웠습니다.
- 디자이너, PM, 백엔드 개발자 등 다양한 직군과 협업하면서 직군별 의사결정 우선순위 차이를 좁히는 커뮤니케이션 방식을 익혔습니다.

캠퍼스 SW 아카데미 TABA 빅데이터 분석 및 인공지능 처리를 위한 SW융합개발자양성

소속/기관: 과학기술정보통신부

활동 연도: 2023

- 프론트엔드·백엔드 개발 및 Figma 기반의 기획 등 전반적인 개발 협업 과정을 경험했습니다.
- 협업 톨과 개발 전반을 아우르며 실제 프로젝트에서의 기본적인 협업 프로세스를 이해했습니다.
- 과정 수료 후 프로젝트 부분에서 우수상 및 우수교육생으로 선정되었습니다.

EDUCATION

- 단국대학교 컴퓨터공학과 : 2019.03. ~ 2026.02. (졸업)

STACKS

Front-End

- HTML, CSS(SCSS), JS(ES6) & TS
- React.js, Next.js
- tailwind
- Redux Toolkit, react-query, zustand
- D3.js

Collaboration & Tools

- slack
- Figma

- Git, Github

PROJECTS

MindGraph (AI 지식 캡처 & 그래프 시각화)

서비스 개요: ChatGPT·Gemini·Claude·Grok의 답변을 캡처 즉시 의미 단위로 묶어 D3.js로 시각화하는 Chrome Extension·Next.js 웹앱입니다.

팀원: 1명 (1인 개발, 도메인 getmindgraph.com 등록 후 출시 전)

기술 스택: TypeScript, Next.js 16 App Router, React 19, D3.js, Tailwind CSS, Chrome Extension Manifest V3, IndexedDB, Supabase, next-intl, GitHub Actions

- D3.js 직접 DOM 조작과 Next.js SSR 충돌을 막기 위해 그래프 엔진을 'use client' 전용 레이어로 격리하고 reactStrictMode를 비활성화해 zoom 이중 바인딩을 차단했습니다.
- Service Worker·Content Script·bridge 번들을 분리하기 위해 3개 vite 설정(extension·content·bridge)과 Next.js webpack 빌드를 공존시켰습니다.
- 네트워크 단절 시 캡처 노드 유실을 막기 위해 IndexedDB 우선 기록과 Pending Ops 큐 기반 Write-Behind 동기화로 오프라인 CRUD와 온라인 복귀 자동 flush를 구현했습니다.
- 4 LLM의 잦은 DOM 변경에 단일 파일 수정으로 대응하기 위해 답변 선택자를 detector.ts SELECTORS에 모으고 hostname 자동 감지로 통합했습니다.
- MV3 Content Script와 Web 사이 Extension 카트 데이터(저장한 LLM 답변 아이템) 릴레이는 bridge.ts ALLOWED_ORIGINS 화이트리스트로 origin을 검증해 허가되지 않은 origin이 카트 데이터를 가져갈 수 없게 차단했습니다.
- Supabase RLS 정책 직접 설계와 GitHub Actions CI/CD·next-intl 한·영 다국어 라우트를 1인이 구축했습니다.

chatGraph (AI 대화 시각화 플랫폼)

서비스 개요: AI와의 대화를 꼬리물기 형식의 마인드맵 그래프로 시각화하여 비선형 사고 흐름을 보존하는 웹 서비스

팀원: 4명 (FE 2명, BE 2명)

기술 스택: TypeScript, React, Next.js, D3.js, Zustand, Tailwind CSS, TanStack Query, Vitest

- 순환 참조를 해소하기 위해 Feature-First 구조로 features·views·shared를 분리하고, 843줄까지 커진 use-question-tree 혹은 책임 단위로 분해해 355줄로 축소했습니다.
- 새 토픽 생성 시 LLM 응답 대기로 화면이 멈추던 문제를 해결하기 위해 sessionStorage에 프롬프트를 저장하고 temp ID로 즉시 라우팅한 뒤 mutation 성공 시 실제 ID로 교체하는 Optimistic 라우팅을 설계했습니다.
- 사이드바 토픽 클릭 시 로딩 지연을 줄이기 위해 Zustand 스토어에 응답을 프리패치해 useSuspenseQuery의 initialData로 주입하여 라우트 전환 시 빈 화면 없이 데이터가 그려지도록 했습니다.
- D3.js Force Simulation이 React 리렌더 시 SVG 트리와 충돌하던 문제를 해결하기 위해 useEffect cleanup과 `d3.selectAll("*").remove()`를 결합해 데이터 변경 시 SVG를 재구성하도록 했습니다.
- 비대해진 페이지 컴포넌트를 TopicContentLayout·StandardChatView·OptimisticChatView로 분리하고 useSuspenseQuery 분기에 맞춰 Suspense 경계를 재설계했습니다.
- 경로 탐색·토픽 토글·새 토픽 시작 혹은 Vitest 회귀 테스트를 작성해 리팩토링 전후 결과 일치를 자동 검증했습니다.

Fit (사일런트 컨퍼런스 플랫폼)

서비스 개요: 무선 헤드폰을 통해 다중 채널의 발표를 골라 듣는 사일런트 컨퍼런스의 온라인/오프라인 하이브리드 중계 플랫폼

팀원: 5명 (FE 3명, BE 2명)

기술 스택: Vite, React, WebRTC, STOMP, SockJS, Framer Motion, Redux Toolkit, redux-persist, Tailwind CSS

- 수백 ms 단위의 저지연 오디오 전송을 위해 RTCPeerConnection API로 발표자·청중 양측 시그널링 클라이언트를 구현하고 STOMP 토픽 위에서 SDP와 ICE 후보를 교환하는 1:N 오디오 송수신 흐름을 설계했습니다.
- SDP 협상 전 도착한 ICE 후보 유실로 연결 실패가 반복되던 문제를 해결하기 위해 ref에 ICE 큐를 두고 setRemoteDescription 직후 일괄 flush하는 버퍼링 로직을 적용해 PeerConnection 성공률을 높였습니다.
- 모바일 망과 행사장 Wi-Fi의 일시 단절에 대비해 STOMP 클라이언트에 reconnectDelay 5초·4초 간격 heartbeat를 설정하고 자체 TURN 서버를 ICE 후보군에 추가해 NAT 환경 연결률을 유지했습니다.
- 시그널링 트래픽과 혼잡도 데이터의 채널 점유 충돌을 막기 위해 혼잡도 전용 STOMP 클라이언트를 별도 구성해 `/sub/session` 토픽만 구독시키고, 채널 간 인원 쓸림을 실시간으로 화면에 반영했습니다.
- 인증 토큰과 좋아요 세션을 Redux Toolkit으로 관리하면서 redux-persist whitelist에 auth만 포함해 새로고침 후 세션은 유지하되 휘발성 데이터가 영속화되는 부작용을 차단했습니다.
- 백엔드 팀과 STOMP `Bearer` 헤더 인증 규약과 토픽 네이밍을 합의해 시그널링 연동 방식을 맞췄습니다.

xAB (A/B 테스트 기반 SNS)

서비스 개요: 두 가지 선택지에 대한 실시간 투표 및 댓글 토론이 가능한 소셜 네트워크 서비스

팀원: 4명 (FE 4명)

기술 스택: TypeScript, Next.js, React Query, Zustand, Supabase, Tailwind CSS

- 메인 피드 초기 로딩 지연을 해결하기 위해 TanStack Query의 `useInfiniteQuery` 와 Intersection Observer 트리거를 결합해 페이지 단위 분할 호출과 지연 로딩이 적용된 무한 스크롤을 구현했습니다.
- 팔로워·팔로잉 목록을 모달로 띄울 때 페이지 맥락이 끊기던 문제를 해결하기 위해 App Router의 병렬 라우트(`@modal`)와 인터셉팅 라우트(`(.)followers` · `(.)followings`)를 결합해 현재 화면 위에 목록 모달이 겹쳐 뜨도록 구현했습니다.
- 팔로우 도메인을 폴스택으로 맡아 팔로우 토글·팔로워·팔로잉 목록 페이지와 `api/follow/[id]` · `api/profile/[id]/followers` · `api/profile/[id]/followings` 라우트를 설계해 팔로우 관계 조회와 변경 흐름을 구현했습니다.
- 프로필 도메인의 헤더·목록 컴포넌트와 `profile/[id]` 페이지를 구현하고 투표·좋아요 API(`api/vote` · `api/like`)를 작성해 사용자 활동 데이터의 조회·갱신 경로를 담당했습니다.